

Brute Force

Print Apple Division

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <numeric>
#include <vector>

#define ll long long

using namespace std;

int main(int argc, char *argv[]) {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int n;
    cin >> n;
    vector<ll> a(n);
    for (auto &x : a) {
        cin >> x;
    }

    int m = n / 2;
    vector<ll> L, R;

    for (int i = 0; i < (1 << m); i++) {
        ll s = 0;
        for (int j = 0; j < m; j++) {
            if (i >> j & 1) {
                s += a[j];
            }
        }
        L.push_back(s);
    }

    for (int i = 0; i < (1 << (n - m)); i++) {
        ll s = 0;
        for (int j = 0; j < n - m; j++) {
            if (i >> j & 1) {
                s += a[m + j];
            }
        }
        R.push_back(s);
    }
}
```

```

sort(R.begin(), R.end());
ll tot = accumulate(a.begin(), a.end(), 0LL);
ll best = LLONG_MAX;

for (ll x : L) {
    ll need = tot / 2 - x;
    auto it = lower_bound(R.begin(), R.end(), need);
    if (it != R.end()) {
        best = min(best, abs(tot - 2 * (x + *it)));
    }
    if (it != R.begin()) {
        best = min(best, abs(tot - 2 * (x + *prev(it))));
    }
}

cout << best << "\n";

return 0;
}

```

Print Apple Division - DP

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<ll> a(n);
    ll S = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        S += a[i];
    }

    const ll DP_THRESHOLD =
        50'000'000LL; // sum threshold for DP (tunable; uses ~O(S) memory)

    vector<char> chosen(n, 0); // will mark items in group A

    if (S <= DP_THRESHOLD) {
        // Dynamic programming pseudo-polynomial
        int Sz = (int)(S + 1);
    }

```

```

vector<char> dp(Sz, 0);
vector<int> parent(Sz, -1); // parent[s] = index of item used to reach s; -1
                             // = unreachable; -2 = base 0

dp[0] = 1;
parent[0] = -2;
for (int i = 0; i < n; ++i) {
    ll w = a[i];
    for (ll s = S; s >= w; --s) {
        if (!dp[(size_t)s] && dp[(size_t)(s - w)]) {
            dp[(size_t)s] = 1;
            parent[(size_t)s] = i;
        }
    }
}

ll target = S / 2;
ll best = 0;
for (ll s = target; s >= 0; --s) {
    if (dp[(size_t)s]) {
        best = s;
        break;
    }
}

// reconstruct
ll cur = best;
while (cur > 0) {
    int idx = parent[(size_t)cur];
    if (idx < 0)
        break; // safety
    chosen[idx] = 1;
    cur -= a[idx];
}

// build groups and print
vector<ll> A, B;
ll sumA = 0, sumB = 0;
for (int i = 0; i < n; ++i) {
    if (chosen[i]) {
        A.push_back(a[i]);
        sumA += a[i];
    } else {
        B.push_back(a[i]);
        sumB += a[i];
    }
}

cout << llabs(sumA - sumB) << '\n';
// print groups (weights). Format: first line is group A items, second line
// group B items. (You can change output formatting if you prefer indices or
// counts.)
for (size_t i = 0; i < A.size(); ++i) {
    if (i)
        cout << ' ';
    cout << A[i];
}
cout << '\n';

```

```

for (size_t i = 0; i < B.size(); ++i) {
    if (i)
        cout << ' ';
    cout << B[i];
}
cout << '\n';
return 0;
}

```

Suffix and Bounding

```

// suffix linear

// lexicographic order for pairs
inline bool leq(int a1, int a2, int b1, int b2) {
    return (a1 < b1 || a1 == b1 && a2 <= b2);
}

// and triples
inline bool leq(int a1, int a2, int a3, int b1, int b2, int b3) {
    return (a1 < b1 || a1 == b1 && leq(a2, a3, b2, b3));
} // and triples

// stably sort a[0..n-1] to b[0..n-1] with keys in 0..K from r
static void radixPass(int *a, int *b, int *r, int n,
                     int K) { // count occurrences
    int *c = new int[K + 1]; // counter array
    for (int i = 0; i <= K; i++)
        c[i] = 0; // reset counters
    for (int i = 0; i < n; i++)
        c[r[a[i]]]++; // count occurrences
    for (int i = 0, sum = 0; i <= K; i++) // exclusive prefix sums
    {
        int t = c[i];
        c[i] = sum;
        sum += t;
    }
    for (int i = 0; i < n; i++)
        b[c[r[a[i]]]++] = a[i]; // sort
    delete[] c;
}

// find the suffix array SA of s[0..n-1] in {1..K}^n
// require s[n]=s[n+1]=s[n+2]=0, n>=2
void suffixArray(int *s, int *SA, int n, int K) {
    int n0 = (n + 2) / 3, n1 = (n + 1) / 3, n2 = n / 3, n02 = n0 + n2;
    int *s12 = new int[n02 + 3];
}

```

```

s12[n02] = s12[n02 + 1] = s12[n02 + 2] = 0;
int *SA12 = new int[n02 + 3];
SA12[n02] = SA12[n02 + 1] = SA12[n02 + 2] = 0;
int *s0 = new int[n0];
int *SA0 = new int[n0];
// generate positions of mod 1 and mod 2 suffixes
// the "(n0-n1)" adds a dummy mod 1 suffix if n%3 == 1
for (int i = 0, j = 0; i < n + (n0 - n1); i++)
    if (i % 3 != 0)
        s12[j++] = i;
// lsb radix sort the mod 1 and mod 2 triples
radixPass(s12, SA12, s + 2, n02, K);
radixPass(SA12, s12, s + 1, n02, K);
radixPass(s12, SA12, s, n02, K);
// find lexicographic names of triples
int name = 0, c0 = -1, c1 = -1, c2 = -1;
for (int i = 0; i < n02; i++) {
    if (s[SA12[i]] != c0 || s[SA12[i] + 1] != c1 || s[SA12[i] + 2] != c2) {
        name++;
        c0 = s[SA12[i]];
        c1 = s[SA12[i] + 1];
        c2 = s[SA12[i] + 2];
    }
    if (SA12[i] % 3 == 1)
        s12[SA12[i] / 3] = name; // left half
    else
        s12[SA12[i] / 3 + n0] = name; // right half
}
// recurse if names are not yet unique
if (name < n02) {
    suffixArray(s12, SA12, n02, name);
    // store unique names in s12 using the suffix array
    for (int i = 0; i < n02; i++)
        s12[SA12[i]] = i + 1;
} else // generate the suffix array of s12 directly
    for (int i = 0; i < n02; i++)
        SA12[s12[i] - 1] = i;
// stably sort the mod 0 suffixes from SA12 by their first character
for (int i = 0, j = 0; i < n02; i++)
    if (SA12[i] < n0)
        s0[j++] = 3 * SA12[i];
radixPass(s0, SA0, s, n0, K);
// merge sorted SA0 suffixes and sorted SA12 suffixes
for (int p = 0, t = n0 - n1, k = 0; k < n; k++) {
#define GetI() (SA12[t] < n0 ? SA12[t] * 3 + 1 : (SA12[t] - n0) * 3 + 2)
    int i = GetI(); // pos of current offset 12 suffix
    int j = SA0[p]; // pos of current offset 0 suffix
    if (SA12[t] < n0
        ? // different compares for mod 1 and mod 2 suffixes
        leq(s[i], s12[SA12[t] + n0], s[j], s12[j / 3])
        : leq(s[i], s[i + 1], s12[SA12[t] - n0 + 1], s[j], s[j + 1],
            s12[j / 3 + n0])) { // suffix from SA12 is smaller
        SA[k] = i;
        t++;
    }
}

```

```

        if (t == n02) // done --- only SA0 suffixes left
            for (k++; p < n0; p++, k++)
                SA[k] = SA0[p];
    } else { // suffix from SA0 is smaller
        SA[k] = j;
        p++;
        if (p == n0) // done --- only SA12 suffixes left
            for (k++; t < n02; t++, k++)
                SA[k] = GetI();
    }
}
delete[] s12;
delete[] SA12;
delete[] SA0;
delete[] s0;
}

int main() {

    // MIGHT NOT WORK FOR EMPTY STRINGS AND 1 CHARACTER STRINGS
    return 0;
}

// ----

// bounding box code.

// Function to calculate a dynamic bounding box based on input points.
Point calculate_bounding_box(const std::vector<Point> &points) {
    long double maxX = -inf, maxY = -inf;
    long double minX = inf, minY = inf;

    // Traverse all points to find the bounding box
    for (const Point &pt : points) {
        if (pt.x > maxX)
            maxX = pt.x;
        if (pt.x < minX)
            minX = pt.x;
        if (pt.y > maxY)
            maxY = pt.y;
        if (pt.y < minY)
            minY = pt.y;
    }

    // Use the most extreme points to define the bounding box
    Point bbox((maxX - minX) * 2,
               (maxY - minY) * 2); // Making the box larger for stability
    return bbox;
}

```